

Manual Técnico: Arquitectura de Chatbot con FastAPI y Google Gemini

Edwin Stiven Pasto

Julián Romero Bocanegra

David Santiago Martín Méndez.

John Harold Perez Calderon

Fundación Universitaria Compensar

Mayo 2026

Título	
Introducción	(Pag. 3)
Objetivo general	(Pag. 3)
Arquitectura de la Solución	(Pag. 3)
Componentes y sus responsabilidades	(Pag. 4)
Tecnologías utilizadas	(Pag. 4)
Configuración e Instalación	(Pag. 5)
Explicación detallada de los servicios	(Pag. 6)
Proceso de puesta en marcha	(Pag. 8)
Verificación del funcionamiento	(Pag. 8)
Manejo de errores comunes	(Pag. 9)
Mantenimiento y monitoreo	(Pag. 9)
Comandos útiles	(Pag. 10)
Recomendaciones de operación	(Pag. 10)
Posibles mejoras futuras	(Pag. 10)
Repositorio original	(Pag. 10)

Introducción

Este documento describe técnicamente la solución implementada para un servidor de chatbot desplegado en entorno Linux (Ubuntu Server). El sistema utiliza una arquitectura desacoplada Cliente-Servidor, donde el backend (construido con FastAPI) gestiona de forma segura y asíncrona la comunicación con la API de Google Gemini. El frontend es una interfaz ligera en la terminal que permite al usuario final interactuar con el modelo de inteligencia artificial.

Objetivo general

Proveer una solución funcional, documentada y portable de un asistente conversacional que pueda ejecutarse en una máquina virtual o en un servidor Ubuntu nativo, garantizando persistencia de conversaciones, supervisión de procesos y fácil mantenimiento.

Arquitectura de la Solución

Diagrama de flujo de información

El sistema está compuesto por cinco módulos principales que interactúan de la siguiente manera:

1. Cliente (`modelo_chatbot.py`) : Envía el mensaje del usuario al servidor mediante una petición HTTP POST.
2. Servidor Backend (`chatbot.py`): Recibe la petición, estructura el payload para Gemini e integra el historial.
3. Gestor de Historial (`chat_history.py`): Registra cada mensaje de usuario y cada respuesta del chatbot en un archivo de texto plano (`historial.txt`).

4. Supervisor (`supervisor.py`): Vigila el proceso del servidor y lo reinicia automáticamente si falla o se detiene.

5. Almacenamiento local : Archivo `historial.txt` que persiste toda la conversación.

Componentes y sus responsabilidades

Componente	Archivo	Rol técnico
Cliente	modelo_chatbot.py	Interfaz de usuario en consola; envía mensajes vía HTTP a <code>http://<IP>:8000/chat</code>
Servidor API	chatbot.py	Endpoint /chat (POST) que orquesta la llamada a Gemini y el registro del historial
Historial	chat_history.py	Funciones <code>guardar_mensaje()</code> y <code>cargar_historial()</code> para persistencia
Supervisor	supervisor.py	Bucle que verifica el proceso uvicorn cada N segundos y lo relanza si es necesario
Configuración	.env	Almacena la API_KEY de Google Gemini de forma segura

Tecnologías utilizadas

- Python 3 pip venv (entorno virtual)
- FastAPI – Framework web asíncrono para el backend
- Uvicorn – Servidor ASGI de alto rendimiento
- Requests – Cliente HTTP para consumir Gemini (en el backend)
- python-dotenv – Carga de variables de entorno
- Google Gemini API – Modelo de lenguaje conversacional
- Ubuntu Server – Sistema operativo base (también funciona en modo puente o NAT)

Configuración e Instalación

Requisitos previos

- Ubuntu Server 20.04 o superior (mínimo 1 GB RAM, 1 CPU).
- Acceso a internet para instalar paquetes y consumir la API de Gemini.
- Clave de API de Google Gemini (obtenida desde Google AI Studio).

Pasos de instalación (Fase 1 – Servidor)

```
sudo apt update && sudo apt upgrade -y
```

Instalar editor nano y herramientas base

```
sudo apt install nano -y
```

Ver IP del servidor (necesaria para el cliente)

```
ip a
```

Instalar Python y herramientas

```
sudo apt install python3 python3-pip -y
```

```
sudo apt install python3-venv -y
```

Creación del entorno virtual y código (Fase 2)

```
mkdir ubuntu_chatbot && cd ubuntu_chatbot
```

```
python3 -m venv venv
```

```
source venv/bin/activate
```

```
pip install fastapi uvicorn python-dotenv requests
```

A continuación, crear los siguientes archivos dentro del directorio `ubuntu\_chatbot`:

Archivo	Contenido (resumen)
.env	API_KEY="tu_clave_aqui"
chatbot.py	Código del servidor FastAPI con endpoint /chat
modelo_chatbot.py	Cliente de consola que envía mensajes
chat_history.py	Funciones para leer/escribir historial.txt
supervisor.py	Script de monitoreo y reinicio del servidor

Explicación detallada de los servicios

Servicio Backend – FastAPI (chatbot.py)

- Expone un endpoint POST en `/chat` que espera un JSON con el campo `"message"`.
- Utiliza la clave API desde `.env` para autenticarse con Gemini.
- Concatena un `SYSTEM\_PROMPT` que define la personalidad del bot (cordial, didáctica).
- ****Middleware de historial****: Antes de llamar a Gemini, guarda la pregunta del usuario; después, guarda la respuesta.
- Retorna la respuesta de Gemini en formato JSON.

Fragmento conceptual:

```
python
```

```
@app.post("/chat")
```

```
async def chat(request: ChatRequest):
```

```
    guardar_mensaje("Usuario", request.message)
```

```
    respuesta = llamar_gemini(request.message)
```

```
    guardar_mensaje("Chatbot", respuesta)
```

```
    return {"response": respuesta}
```

Cliente de Consola (`modelo\_chatbot.py`)

- Se ejecuta en una terminal independiente.
- Solicita al usuario que ingrese mensajes (bucle infinito hasta escribir `salir`).
- Envía cada mensaje al servidor en `http://<IP\_del\_servidor>:8000/chat`.
- Muestra la respuesta del chatbot y permite finalizar con `CtrlC` o escribiendo `salir`.

Gestor de Historial (`chat\_history.py`)

- Abre el archivo `historial.txt` en modo append.
- Escribe líneas con formato: `timestamp - Rol: mensaje`.
- Permite cargar el historial completo (útil para futuras mejoras de contexto).

Supervisor de Procesos (`supervisor.py`)

- Se ejecuta también dentro del entorno virtual.
- Comprueba si el proceso `uvicorn` está activo mediante `ps aux | grep "uvicorn chatbot:app"`.

- Si no encuentra el proceso, lo lanza automáticamente.
- Ideal para mantener el servicio activo ante fallos inesperados.

Proceso de puesta en marcha

Preparación de la red (máquina virtual)

- Modo Puente (Bridged): La VM obtiene una IP propia en la red local. El cliente usará esa IP directamente.
- Modo NAT con reenvío de puertos: Mapear el puerto 8000 de la VM al puerto 8000 del host.

Inicio de servicios (tres terminales)

Terminal	Comando
1 (Backend)	<code>cd ~/ubuntu_chatbot</code>
	<code>source venv/bin/activate</code>
	<code>uvicorn chatbot:app --host 0.0.0.0 --port 8000</code>
2 (Supervisor)	<code>cd ~/ubuntu_chatbot</code>
	<code>source venv/bin/activate</code>
	<code>python3 supervisor.py</code>
3 (Cliente)	<code>cd ~/ubuntu_chatbot</code>
	<code>source venv/bin/activate</code>
	<code>python3 modelo_chatbot.py</code>

Verificación del funcionamiento

1. Desde el cliente, enviar un mensaje de prueba: `"Hola, ¿cómo estás?"`
2. El servidor debe responder con el texto generado por Gemini.

3. Consultar el historial: ``cat historial.txt`` – se deben ver ambas líneas (usuario y chatbot).

Manejo de errores comunes

``IndentationError: expected an indented block``

- Causa: Python espera código sangrado después de un bloque ``try:``, ``if:``, etc.
- Solución: Revisar que después de ``try:`` haya al menos una línea con sangría (espacios o tabulaciones consistentes).

Error de conexión al servidor

- Verificar que el backend esté corriendo en el puerto 8000.
- Confirmar la IP correcta del servidor con ``ip a``.
- Probar conectividad: ``curl http://<IP>:8000/chat`` (debe devolver ``{"detail":"Method Not Allowed"}`` si no se envía POST).

Clave API inválida

- Revisar que el archivo ``.env`` contenga exactamente ``API_KEY="clave"``.
- Cargar las variables en Python con ``load_dotenv()``.

Mantenimiento y monitoreo

Comandos útiles

- Ver historial completo: ``cat historial.txt``
- Ver procesos activos: ``ps aux | grep uvicorn``
- Matar un proceso colgado: ``kill -9 <PID>``

- Reiniciar manualmente el backend: detener con ``CtrlC`` y volver a ejecutar ``uvicorn``.

Recomendaciones de operación

1. Siempre iniciar el backend antes que el cliente.
2. Ejecutar el supervisor si se desea alta disponibilidad (en entornos productivos).
3. Respalidar el archivo ``historial.txt`` periódicamente si se necesita conservar conversaciones.
4. Monitorear la memoria: Si la VM tiene menos de 1 GB, reducir otros servicios.

Posibles mejoras futuras

- Agregar contexto de historial en cada llamada a Gemini (enviando las últimas N interacciones).
- Implementar autenticación básica en el endpoint ``/chat``.
- Migrar el almacenamiento de historial a una base de datos ligera (SQLite).
- Crear un frontend web básico (HTML/JS) que consuma el mismo endpoint.

Repositorio original

<https://github.com/Julian-Romero-Bocanegra/SERVIDOR-CHATBOT>